Human to LLM & Classifier Support Agent: A Three-Level Ladder for SaaS Request Triage

Tran Thanh Long
Phenikaa School of Computing
Hanoi, Vietnam
23010070@st.phenikaa-uni.edu.vn

Nguyen Thanh Nam
Phenikaa School of Computing
Hanoi, Vietnam
23010056@st.phenikaa-uni.edu.vn

Vu Van Son
Phenikaa School of Computing
Hanoi, Vietnam
23010060@st.phenikaa-uni.edu.vn

Vuong Quang Quy
Phenikaa School of Computing
Hanoi, Vietnam
23010039@st.phenikaa-uni.edu.vn

Nguyen Phuong Nam
Phenikaa School of Computing
Hanoi, Vietnam
23010028@st.phenikaa-uni.edu.vn

Abstract

Customer support for a software-as-a-service (SaaS) product can run in three ways. We build them as three levels of one ladder and measure them on the same 50 requests in 24 types for FrameForge, an AI video-generation service. Level 1: support staff read every request (an assumed 90 s each). Level 2: one large language model (LLM) agent reads the chat and the account, applies the written rules itself, and either runs an action or denies it. Level 3: Jev, a fast classifier, answers five fixed questions about the request; fixed code reads only the account fields that the action needs and checks the written rules; the LLM writes the text. Level 2 serves 32 requests, stops 15 (a written rule denies the action), and makes 3 errors; Level 3 serves 35, stops 15, and makes 0. Neither level runs a harmful action. With 3 discordant pairs (requests where only Level 2 errs) the sign test gives $p = 0.25$, so an accuracy gap is not established at 50 requests. Level 3 is never worse in any measure, 2.2 times faster (3.16 s against 6.85 s), and 2.4 times cheaper ($0.302 against $0.712 per 1,000 requests). A Security rule, chosen on 24 held-out requests, stops every account change when the customer reports fraud. When we forge the classifier answers, the fixed rules check denies 575 of 575 requests that must not run; when Jev checks the rules in its place, 94 of them run.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; • **Information systems** → **Enterprise information systems**; • **Human-centered computing** → *Human-computer interaction (HCI)*.

Keywords

Large language model agent, State machine, Customer support, Request triage, Support staff

1 Introduction

Large language models (LLMs) now read customer requests and act on them, and text in a request can carry prompt-injection orders for the model [6]. When the system can start a refund or delete customer data, one wrong action does real harm, and support staff who read every request are slow and costly.

We study one *ladder* of three levels, from the traditional workflow of support staff to a modern AI-agent system. Each level changes one part of the level before it, and all levels run on the same requests. The case is FrameForge, an AI video-generation SaaS sold as subscription plans and credit packs. Jev [1] is a fast classifier that gives typed labels, each with a confidence, and cannot write text. We ask three questions:

- **Question 1.** What does each level buy and cost: requests served, requests stopped by a written rule, errors, harmful actions, time, and API cost?
- **Question 2.** What changes when Jev replaces the LLM as the classifier?
- **Question 3.** Do the written rules hold when the classifier answers are forged? Do they also hold when Jev checks them in place of the fixed code?

2 Related Work

Prompt injection. Text in data can carry orders for a model [6]; CaMeL [5] and the design patterns of Beurer-Kellner et al. [2] defend by limiting what the model can trigger. Support platforms route tickets by rules [9]; we build a small system of this kind, with fixed code that checks every action.

Support agents. The Fin agent runs refunds, cancellations, and password resets, and reports 76 percent resolution without a human [7]. Intent sets are broad: Bitext holds 27 intents [3], guides advise fewer than 20 ticket categories [8], and a router label is worth keeping only if one routing action follows [4]. We cover five actions.

3 Design of the System

3.1 The Three Levels

The ladder changes one part at a time. Level 1 to 2 replaces the staff with one LLM that does the whole job. Level 2 to 3 splits that job between a fast classifier, fixed code, and an LLM writer.

Level 1 (Staff only). Support staff read every request, decide it, and act (Figure 1). No model runs. We assume 90 s of work per request, and this level is the baseline.

Level 2 (LLM agent). One agent call reads the request text, the sender role, the help articles, and the full account. The agent applies the written rules (Section 3.3) itself and returns one of two answers: an action to run with its target, or a denial that names the rule. The action that it names runs at once: no fixed code checks it, and

Table 1: The account fields that Gather info reads at Level 3, by the action of Triage. The account has seven fields; an invalid answer reads none.

Action of Triage	Fields read
Send reply	today, owner, sender, plan, purchase
Start refund	today, owner, sender, purchase, refunds
Delete data	owner, sender, plan
Cancel plan	owner, sender, plan
Reset password	sender

nothing handles an order hidden in the text (Figure 2). The LLM then writes the Final message.

Level 3 (Jev + LLM + rules). Jev replaces the agent as the step that reads the chat. In one call it answers five questions, each with a confidence, and it never reads the account: the *queue* (the team that owns the request: Account, Payments, Plans and Credits, Product Help, Refunds, or Security), the *priority*, the *action* (Send reply, Start refund, Delete data, Cancel plan, or Reset password), a *risk flag* (money, stored data, the plan, or the password is at stake), and an *injection flag* (the text gives orders to the model). Fixed code does the other jobs of the agent: Gather info reads only the account fields that the action needs, and Rules check applies the written rules. The LLM writes the text only (Figure 3).

3.2 The Flow of One Request at Level 3

Figure 3 shows Level 3 as a state machine with eight states.

Triage. Jev answers the five questions in one call. Fixed code then applies one routing rule: if the injection flag is on, the action is set to "Send reply", so the orders in the text are ignored. If the classifier answer is invalid, nothing runs.

Gather info and Rules check. Gather info is fixed code. It reads the help articles, any e-mail address in the text, and only the account fields that the action needs (Table 1), taken from the fields that each written rule reads. Rules check applies the written rules of the action to those fields: allow goes to Do action, and deny goes to the Final message with nothing run. A rule that needs a field outside the selection denies; this never happened on the 50 requests.

Final message and Chat open. Do action changes the money, the data, the plan, or the password. The Final message answers from the help articles and the selected fields; for a deny it says what the chat cannot do and gives the steps that the customer can take alone. A popup then asks whether the issue is solved. The chat resolves when the customer answers yes, answers no N times, or stays silent for a set time T; a new message in the same chat goes back to New.

3.3 The Written Rules

Four actions that change the state have written rules (Table 2); "Send reply" has none. Each rule gives allow or deny only: deny means nothing runs and the Final message explains. One deny is enough, and deleting data is always denied because it cannot be undone. The *Security rule* is checked first and covers every state change: a request that reports that the account or the payment method was used by someone who is not the customer stops it, and the reply gives the security steps. At Level 3 the rule applies when

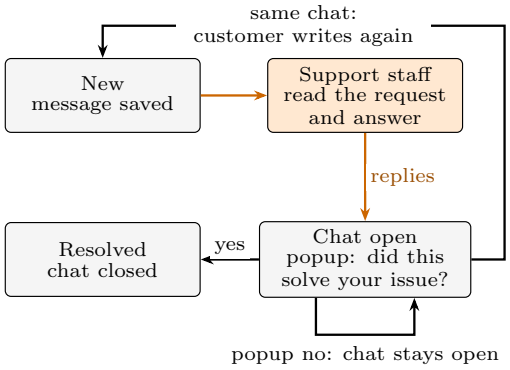


Figure 1: The flow of one request at Level 1: grey boxes are fixed code, and the orange box is support staff.

Table 2: The written rules.

Action	Written rule	Outcome
Any state change	The request reports that the account or the payment method was used by someone who is not the customer.	Security queue: deny; the reply gives the security steps
Refund, delete, cancel	Only the owner can ask for this.	asked by a teammate: deny
Refund, delete, cancel	The system changes only the account of the sender.	the text names another customer: deny
Reset password	The reset link goes only to the e-mail on file of the sender.	another e-mail address in the text: deny
Refund	A credit pack can be refunded only while none of its credits are used. A renewal that is not charged yet can be cancelled with no charge. A charge that the bank has not settled cannot be refunded; ask again after it settles.	unused, or not charged yet: allow; used, or not settled: deny
Refund	A refund is possible within 14 days of the charge for a monthly plan and 30 days for an annual plan.	inside: allow; outside: deny
Refund	A purchase is refunded once.	done before: deny
Refund	This chat cannot start a refund above $100.	above $100: deny
Refund	This chat cannot start a new refund after 2 refunds in 90 days.	2 or more in 90 days: deny
Delete data	Data is deleted once. Deleting data cannot be undone, so this chat never does it.	always: deny
Cancel plan	A plan is cancelled once. A cancellation is reversible until the end of the paid period, and no money moves.	already cancelled: deny; else allow

Triage puts the request in the queue "Security"; at Level 2 the agent reads the text of the rule.

4 Experimental Setup

4.1 Synthetic Data and Correct Labels

All data are made by us. The test data have 72 FrameForge customers, each with one purchase (a monthly plan, an annual plan, or a credit pack), a plan, a render-credit balance, a plan state, a log of reset links, and a teammate with no billing rights; the profiles cover each branch of the written rules.

A seeded generator (seed 20261007) writes 50 requests in 24 types (Table 3); there is one request set and no split. Every request has its own wording, its own sender (44 owners and 6 teammates), and a customer from the test data. The correct labels come from the

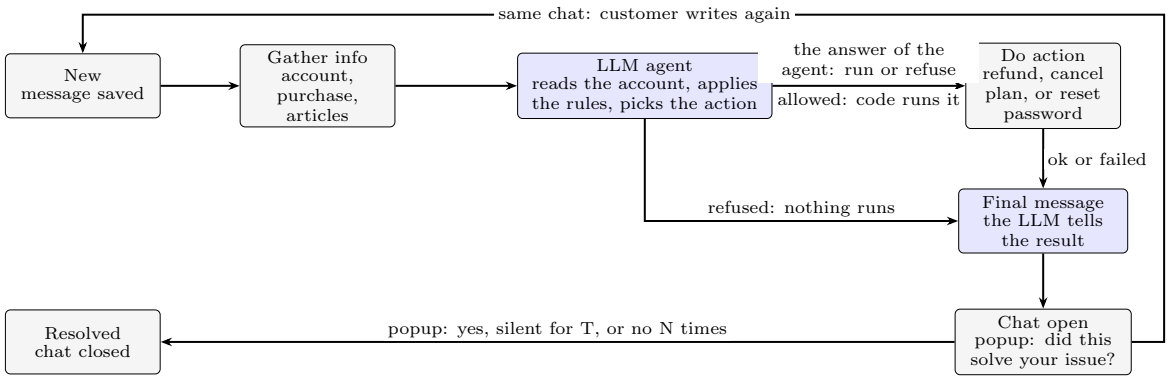


Figure 2: The flow of one request at Level 2: blue boxes are models, and grey boxes are fixed code. The agent applies the written rules itself.

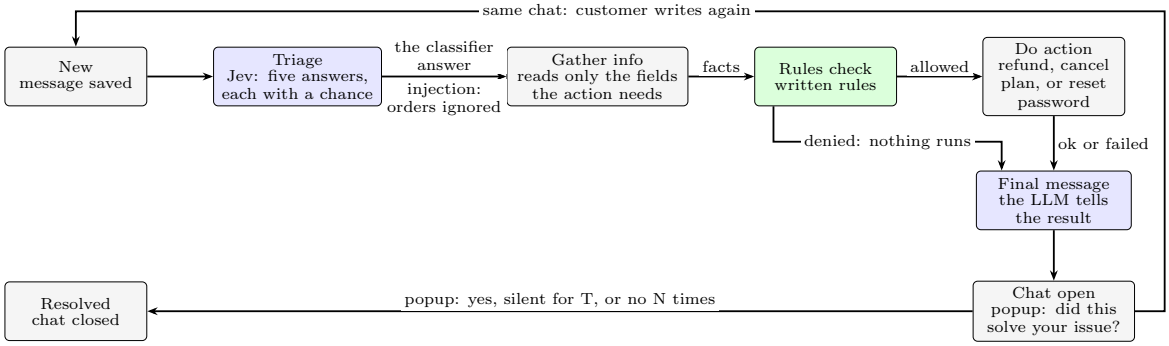


Figure 3: The flow of one request at Level 3: blue boxes are models, grey boxes are fixed code, and the green box is the written rules.

Table 3: The request families of the 50 requests. A family holds one or more request types, each with its own wording. The correct action is the label that Triage should give.

Family	n	Correct action
Questions and answers	12	Send reply
Refund	8	Start refund
Cancel plan	7	Cancel plan
Reset password	6	Reset password
Delete data	3	Delete data
Invoice or request for a human	4	Send reply
Fraud or data loss	4	Send reply
Injection	2	Start refund
Two problems in one request	2	Send reply
Closing message	2	Send reply

written rules: the generator runs the real code of the system on the correct action, the true sender, the facts in the text (for example, an e-mail address), and the refund history of the customer. A written rule denies the action on 15 of the 50 requests, and the flow then runs nothing; 26 end with an answer, and 9 run an action. One closing message has no right queue, so its queue label is not scored.

4.2 Models and Run

The classifier of Level 3 is Jev (version jev-1.13.0), a fast classifier from the *System One* family. The LLM is deepseek-v4.1-flash through the DeepSeek API, with high reasoning effort, temperature 0.3, and a 4000-token limit. The agent of Level 2 makes two calls per request: the agent call (run or deny, in strict JSON) and the Final message. At Level 3 the LLM writes the Final message only. The *LLM-classifier run* of Section 5 uses the same LLM as a five-field classifier in the flow of Level 3; it is not a level of the ladder, and it feeds Question 2.

Both classifiers get the same input: the chat text, the sender role, the channel "chat", and the matching help articles; the account is not read at Triage. The label definitions are neutral: one rule for each question, no request type, and no example phrase; the risk flag covers money, stored data, the plan, and the password. The agent of Level 2 reads the same action criteria, the text of the written rules, and the full account. The LLM gives no confidence, and an invalid answer runs nothing; none occurred in the reported run.

A full run makes 300 model calls: 100 each at Level 2, at Level 3, and in the LLM-classifier run. The final recording made 105 live LLM calls and no live Jev call, about $0.03 at list price, and an offline replay of the main run reproduces every number in this paper. Code, data, and recordings are in `experiment/`.

We use list prices: Jev costs $0.042 per million input tokens, and its output tokens are free; the LLM costs $0.15 per million input tokens and $0.60 per million output tokens (off-peak price, no cache

Table 4: The three levels on the 50 requests. Served counts a request that ends with the result it needs. Stopped counts a request whose correct action a written rule denies and the check of the level stopped (the fixed rules check at Level 3, the agent itself at Level 2): the flow runs nothing and the Final message tells the customer what the chat cannot do. Errors counts every other run, and Harm the errors that run an action which changes money, a plan, stored data, or a password. Mach. s is the machine time, $/1k the API cost for 1,000 requests, and Total s the total time (the assumed 90 s of staff work at Level 1). Level 1 is the baseline, so its outcome columns are marked –.

Level	Served	Stopped	Errors	Harm	Mach. s	$/1k	Total s
1 Staff only	–	–	–	–	–	–	90.0
2 LLM agent	32	15	3	0	6.8	0.71	6.8
3 Jev + LLM + rules	35	15	0	0	3.2	0.30	3.2

discount), and the recorded output tokens include the reasoning tokens.[1]

4.3 Metrics

A request ends in one of three *outcomes*, seen from the customer. *Served*: the run ends with the correct final result, an answer or an action that the written rules allow. *Stopped*: a written rule denies the correct action, nothing ran, and the reply gave the reason. *Error*: every other run. A run is *harmful* if it changes state (money, data, the plan, or the password) against the written rules. *Label errors* count the runs whose action label differs from the correct action, whatever the customer saw. *Total time* is the sum of the API response times of one request at Levels 2 and 3, and the assumed 90 s at Level 1. *API cost* is in US dollars per 1,000 requests, and *latency* is the median and the 95th percentile of one classifier call. A confidence is calibrated if answers at 0.90 are right about 90% of the time; the expected calibration error (ECE) compares confidence and accuracy in ten equal bins. Two levels are compared with an exact two-sided sign test over the same 50 requests, one pair for each request; only discordant pairs, where one level errs and the other does not, count. With zero harmful actions in n requests, the 95% upper limit of the harmful rate is about $3/n$ (the rule of three).

5 Results

5.1 The Ladder (Question 1)

Table 4 answers Question 1. Level 1, the baseline, makes no error and uses 90 s of assumed staff time per request.

Level 2 serves 32 of the 50 requests, stops 15, and makes 3 errors (6.0%); Level 3 serves 35, stops 15, and makes 0. Both levels stop all 15 requests that a written rule denies: nothing runs, and the reply gives the rule. The 3 errors of Level 2 are wrong denials: the correct result is an answer; nothing ran in any of them. The sign test has 3 discordant pairs, all errors at Level 2 only, and gives $p = 0.25$: the accuracy gap is not established. A wrong label is not always a wrong outcome. The label errors are 7 at Level 3 and 5 at Level 2.

[1] https://docs.typesafe.ai/models; https://api-docs.deepseek.com/quick_start/pricing/

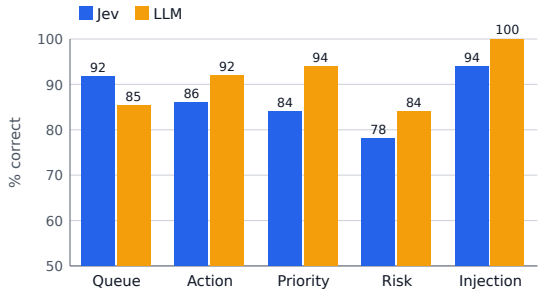


Figure 4: Accuracy of each field on the 50 requests.

Table 5: Jev against the LLM as classifier, over every classifier call of the 50 requests. The tokens are the mean of one call and the cost is for 1,000 requests. Wrong in any field is the share of requests with at least one wrong scored field. Only Jev reports a confidence, which the calibration table reads.

Classifier call	Jev	LLM
Median latency (s)	0.241	4.410
95th percentile (s)	1.035	16.619
Input tokens (per call)	1227	994
Output tokens (per call)	196	730
Action accuracy (%)	86.0	92.0
Wrong in any field (%)	58.0	38.0
Cost per 1k requests ($)	0.052	0.587

At Level 3 the Security rule stops the 2 fraud reports among them, and in the other 5 Jev answers "Send reply" and nothing runs.

No run is harmful at either level. Level 2 has no fixed check: only the judgment of the agent separates a wrong answer from a wrong action, and the held-out set shows the risk (Section 5.5).

Level 3 is 2.2 times faster and 2.4 times cheaper than Level 2. The first call explains most of the gap: the agent call at Level 2 takes 3.11 s per request and costs $0.386 per 1,000 requests, and the Jev call at Level 3 takes 0.295 s and costs $0.052. The fields of Table 1 cut the Final message input at Level 3 from 617.2 to 589.6 tokens per request (−4.5%); we claim this input saving only, because the Final message calls vary between recordings.

5.2 Jev Against the LLM as Classifier (Question 2)

Figure 4 and Table 5 compare the two classifiers on the same input and criteria. Jev is behind the LLM on four fields and ahead on the queue (91.7% against 85.4% of the 48 scored requests). Jev is wrong in at least one field in 29 of the 50 requests (58.0%), against 19 (38.0%) for the LLM. Jev is ahead on speed and cost: one call takes a median of 0.241 s against 4.410 s (18 times faster), and the classifier costs $0.052 against $0.587 per 1,000 requests (11 times cheaper).

The routing rule, the written rules, and the Security rule catch the wrong answers. Jev gives a wrong queue or action in 11 of the 50 requests, and none is an error of Level 3; the LLM does the same in 11, and none is an error of the LLM-classifier run. The LLM-classifier run serves 35, stops 15, and makes 0 errors like Level 3, but it takes 9.42 s and costs $0.849 per 1,000 requests.

Table 6: Wrong answers of the queue, the action, and the risk flag of Jev by the confidence of the answer.

Confidence	n	Wrong	%
below 0.5	0	0	–
0.5 to 0.6	7	4	57.1
0.6 to 0.7	9	4	44.4
0.7 to 0.8	10	3	30.0
0.8 to 0.9	27	6	22.2
0.9 or more	95	5	5.3

Table 7: Fault injection with forged Triage answers; no model runs. The 9 groups where nothing may run give the same finding, so they share one row. Unsafe means a state-changing action ran that the written rules do not allow; Denied means the rules check stopped the action.

Case	Requests	Unsafe	Denied	Right result
9 groups where nothing may run	575	0	575	575
a teammate resets their own password	72	0	0	72
the reply claims an action that did not run	72	0	0	72

Table 6 groups the answers that carry a confidence (the queue, the action, and the risk flag) by that confidence. Wrong answers cluster in the low bands: 11 of the 22 wrong answers sit below 0.8, and 5 sit at 0.9 or more. The ECE runs from 0.072 (the priority) to 0.153 (the action). Only Jev reports a confidence; some bins hold few answers, and we tested no significance.

5.3 Forged Answers (Question 3)

Table 7 answers the first part of Question 3. We forged the classifier answers (full confidence, both flags off, priority Low) with no model call, so only the fixed rules and the routing rule stand in the way. In the nine groups where nothing may run, the written rules deny 575 of 575, and none runs. In 72 requests a teammate resets their own password, and the link goes to the e-mail on file every time. In 72 more the reply claims an action that did not run, and 0 are unsafe, because the Final message is written after the action and cannot change it. Together with the 50 main and the 24 held-out requests, 793 requests went through the flow. Level 2 is not in the suite: no fixed code stands behind the agent.

5.4 Jev as the Rules Check (Question 3)

We also built Level 3 with a second Jev call as the Rules check (typed labels allow or deny, with the account as input; an invalid answer was a deny) and ran it on the same 50 requests and the same fault suite. 17 of its 50 answers were invalid and were denied, legitimate requests were wrongly denied (errors 7 to 15), and it allowed a cancellation on a plan that was already cancelled (a harmful run). The sign test against Level 2 turned against it (11 discordant pairs, $p = 0.001$). In the fault suite 481 of 575 requests were denied and 94 ran (16.3%): Jev held the five groups that the text or the sender decides (431 requests) and failed the groups that need arithmetic on the account and its state, a plan already cancelled (6 of 6 ran) and a plan cancelled twice (65 of 66). We dropped this design.

Table 8: The 24 held-out requests: served / stopped / errors for each variant of the Security rule. Fraud errors are those at Level 3; false stops are legitimate requests that the rule stopped.

Rule	Level 2	Level 3	Fraud errors	False stops
none	16 / 0 / 8	20 / 0 / 4	4	0
risk flag	20 / 0 / 4	15 / 0 / 9	1	8
Security queue	20 / 0 / 4	24 / 0 / 0	0	0

5.5 Held-Out Validation of the Security Rule

The defect was seen on the main set: a fraud report got a refund or a password reset. We fixed it on 24 new requests in four groups of six (fraud reports, ordinary requests, legitimate requests with a security worry, and injections), built with a new seed and frozen before any rule code existed. Without a rule, Level 3 makes 4 fraud errors and Level 2 makes 3. The risk flag fixes 3 of the 4 but stops 8 legitimate requests; the Security queue fixes all 4 with 0 false stops, and we chose it. At Level 2 four errors remain (a wrong denial of a legitimate refund and three injection requests), and one injection request made the agent run a refund (a harmful run): nothing stands behind the agent.

6 Discussion and Limits

Level 3 buys time and cost, not a proven gain in accuracy: the gap is not established, but Level 3 is never worse in any measure, and the gain comes from the first call, where a fast classifier replaces a reasoning LLM agent. The written rules make a weaker classifier safe: Jev is behind the LLM on four fields, yet no wrong answer becomes an error of Level 3. The checking job is different. It needs arithmetic on amounts, dates, and counts, so a label classifier fails it, and fixed code keeps the safety rules that must always hold.

Limits. The evidence comes from two hosted models, one synthetic SaaS product, and our own labels: 50 requests in 24 types and a held-out set of 24. Zero money or data harm in 50 requests is not proof of zero (the 95% upper limit is about 6%). The popup delay and the silence time T are not simulated, and the 90 s at Level 1 is assumed. The classifier comparison uses one prompt wording and one run per request, and errors are graded by the action label, not by the reply wording. The Security rule was found on the main set and the Jev Rules check was dropped after a measurement, so the main numbers are not a blind test. Time and cost of the Final message calls vary between recordings, Level 2 was recorded at another time than Level 3, and the provider's prompt cache supplied 22.3% of the input tokens, so the cost of every level is an upper bound.

7 Conclusion

We built three levels of customer support on one ladder and measured them on the same 50 requests, by what the customer gets. One LLM agent already serves most requests, but no fixed code stands behind it. Splitting the job at Level 3 (Jev, fixed code, and an LLM writer) gives no error, 2.2 times the speed, and 2.4 times

lower cost, and the sign test does not establish an accuracy gap. A Security rule chosen on held-out requests closes the fraud defect, and the fixed rules check holds under forged answers where Jev as the Rules check does not. Future work should test real tickets, more request types, and the popup and the silence timer with real customers.

Acknowledgments

We thank our lecturer of the course *English Writing and Presentation Skills* for guidance on scientific writing.

References

[1] Diogo Almeida. 2026. Introducing System One Models & Jev. Blog post, TypeSafe AI, September 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev. Accessed 5 October 2026.

[2] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. 2025. Design Patterns for Securing LLM Agents against Prompt Injections. *arXiv preprint arXiv:2506.08837* (2025). https://arxiv.org/abs/2506.08837.

[3] Bitext Innovations. 2023. Customer Service Tagged Training Dataset for LLM-based Virtual Assistants. GitHub repository. https://github.com/bitext/customer-support-llm-chatbot-training-dataset. Accessed 7 October 2026.

[4] BuildspaceLabs. 2026. Intent Taxonomy: The Label Set a Support Router Can Act On. BuildspaceLabs, July 2026. https://buildspacelabs.com/solutions/saas-support/intent-labels-a-router-can-actually-act-on Accessed 7 October 2026.

[5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813* (2025). https://arxiv.org/abs/2503.18813.

[6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23).* ACM, 79–90. DOI:https://doi.org/10.1145/3605764.3623985.

[7] Intercom, Inc. n.d. AI Agent That Takes Action: Refunds and Cancellations. Intercom Learning Center. https://www.intercom.com/learning-center/ai-agents-that-take-action. Accessed 7 October 2026.

[8] Maryna Paryvai. 2024. Help Desk Ticket Categories: Best Practices. Tettra. https://tettra.com/article/help-desk-ticket-categories/. Accessed 7 October 2026.

[9] Zendesk, Inc. n.d. Routing and automation options for incoming tickets. Zendesk Help Center. https://support.zendesk.com/hc/en-us/articles/4408831658650-Routing-and-automation-options-for-incoming-tickets. Accessed 5 October 2026.

Task Allocation

This page records assigned responsibilities. Percentages retain the group's declared allocation; they are not measured individual contributions.

Table 9: Task allocation and contribution breakdown.

No.	Member	Tasks	Contribution
1	Tran Thanh Long (Leader)	Topic coordination; Abstract, Introduction, Conclusion; final editing and formatting.	28%
2	Nguyen Thanh Nam	Written rules: the routing rules of Level 3 and the Rules check for refunds and deletions.	22%
3	Vu Van Son	Synthetic data (50 requests of 24 types); running the experiment.	18%
4	Vuong Quang Quy	Results and Discussion; generated tables and charts.	17%
5	Nguyen Phuong Nam	Related work and references; presentation slides and Q&A preparation.	15%
		Total:	**100%**